Laporan Tugas Kecil 1

IF2211 Strategi Algoritma

Voxelization Objek 3D menggunakan Octree

Semester II Tahun 2025/2026

Eliana Natalie Widjojo/13524116
Varistha Devi/13524135

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

1. Pendahuluan

Voxelization adalah proses konversi objek tiga dimensi menjadi sebuauh model yang tersusun dari kubus-kubus kecil. Proses konversi menggunakan struktur data Octree. Octree adalah sebuah *tree* yang tiap cabangnya terdiri dari delapan anak. Octree digunakan untuk mempartisi ruang tiga dimensi dan membaginya secara rekursif menjadi delapan oktan.

Divide and conquer adalah paradigma algoritma yang menyelesaikan masalah dengan dengan membaginya menjadi tahap *divide* (memecah masalah besar menjadi sub-masalah yang lebih kecil), *conquer* (menyelesaikan setiap sub-masalah secara rekursif), dan *combine* (menggabungkan hasil dari setiap sub-masalah untuk mendapatkan solusi akhir).

2. Penjelasan Algoritma *Divide and Conquer*

Algoritma *divide and conquer* terdapat di fungsi **buildOctree**. Berikut adalah *pseudocode* dari fungsi tersebut:

```
function buildOctree(node: OctreeNode, triangles: list of Triangle, maxDepth:
integer, stats: Stats)
{ Membangun struktur octree dengan metode divide and conquer }

KAMUS LOKAL

    childSize : float
    i : integer
    ox, oy, oz : float
    childMin, childCenter, childHalf : Vector3
    childTriangles : list of Triangle
    child : OctreeNode

ALGORITMA

    lock(stats.mu)
        stats.nodeCount[node.depth] ← stats.nodeCount[node.depth] + 1
    unlock(stats.mu)

    if node.depth = maxDepth then
        node.isLeaf ← true
        return

    childSize ← node.size / 2

    i traversal[0..7]
        ox ← (i >> 0) AND 1
        oy ← (i >> 1) AND 1
        oz ← (i >> 2) AND 1

    childMin.x ← node.minX + ox * childSize
    childMin.y ← node.minY + oy * childSize
    childMin.z ← node.minZ + oz * childSize

    childCenter.x ← childMin.x + childSize / 2
    childCenter.y ← childMin.y + childSize / 2
    childCenter.z ← childMin.z + childSize / 2

    childHalf ← (childSize/2, childSize/2, childSize/2)

    childTriangles ← []

    for setiap tri dalam triangles do
        if isTriBoxOverlap(childCenter, childHalf, tri) then
                append tri ke childTriangles

        if length(childTriangles) = 0 then
                lock(stats.mu)
                stats.skippedCount[node.depth + 1] ←
                    stats.skippedCount[node.depth + 1] + 1
                unlock(stats.mu)
                continue

        child ← newOctNode(childMin.x, childMin.y, childMin.z, childSize,
                node.depth + 1)

        node.children[i] ← child

        if node.depth < 3 then
            parallel buildOctree(child, childTriangles, maxDepth, stats)

        else
            buildOctree(child, childTriangles, maxDepth, stats)
```

1. Langkah 1: *Divide*
   Tahap *divide* adalah tahap di mana kita membagi masalah. Tahap ini terdapat pada bagian:
   ```
   i traversal[0..7]
   ```
   dan
   ```
   childTriangles ← []

   for setiap tri dalam triangles do
       if isTriBoxOverlap(childCenter, childHalf, tri) then
               append tri ke childTriangles
   ```
   Node (ruang besar) dibagi jadi 8 sub-ruang. Setiap triangle dipisahkan ke sub-ruang yang sesuai .

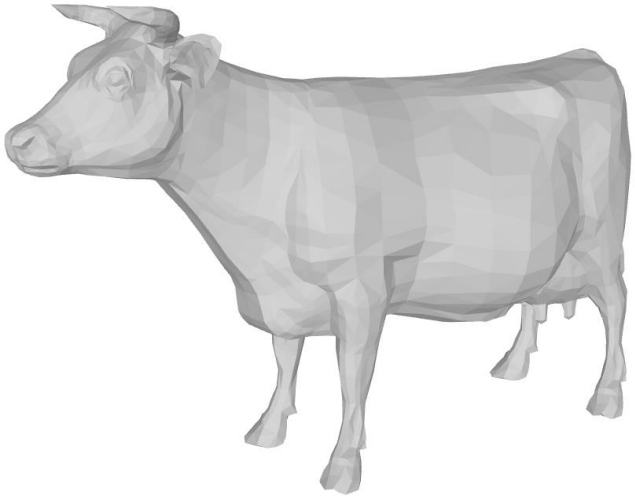
2. Langkah 2: *Conquer*
   Tahap *conquer* adalah tahap menyelesaikan sub-masalah. Tahap ini terdapat pada bagian:
   ```
   buildOctree(child, childTriangles, maxDepth, stats)
   ```
   dan
   ```
   parallel buildOctree(child, childTriangles, maxDepth, stats)
   ```
   Setiap sub-node menyelesaikan masalahnya sendiri dan dilakukan secara rekursif.

3. Langkah 3: *Combine*
   Tahap *combine* adalah tahap menggabungkan hasil. Tahap ini terdapat pada bagian:
   ```
   node.children[i] ← child
   ```
   Artinya adalah hasil dari setiap sub-masalah (child node) digabung dalam struktur tree.

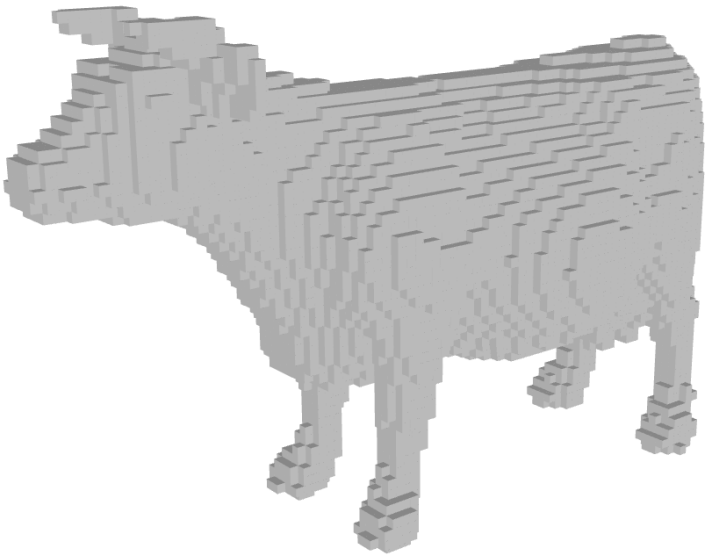
   Kesimpulannya, divide dilakukan dengan cara membagi node menjadi 8 child dan mendistribusikan triangle ke masing-masing child. Conquer dilakukan dengan pemanggilan rekursif buildOctree pada setiap child node. Combine dilakukan dengan menyimpan hasil subtree ke dalam children dari node induk.

3. Uji Coba *Test Case*

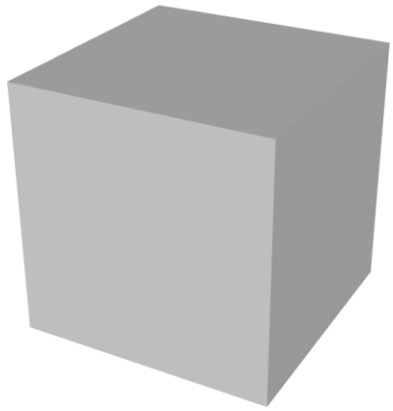
1.

Input File	

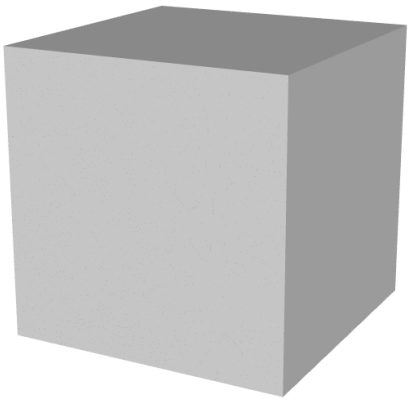
Output CLI	PS D:\stima\tucil 2\Tucil2_13524116_13524135\src> go run . cow.obj 7
	Input file : ..\test\cow.obj
	Output file: ..\test\cow_voxed.obj
	Max depth : 7
	Memproses...
	Parse selesai: 5804 triangles
	Bounding box: min = (-7.232, -8.447, -8.008) size = 16.016
	Membangun octree...
	Octree selesai: 9639 voxel ditemukan
	Menulis output...
	================ Stats ================
	Jumlah voxel : 9639
	Jumlah vertices : 77112
	Jumlah faces : 115668
	Kedalaman octree: 7
	Statistik node per depth:
	  1 : 8
	  2 : 14
	  3 : 40
	  4 : 168
	  5 : 602
	  6 : 2349
	  7 : 9639
	Statistik node yang dilewati per depth:
	  1 : 0
	  2 : 50
	  3 : 72
	  4 : 152
	  5 : 742
	  6 : 2467
	  7 : 9153
	Waktu eksekusi: 556.4622ms
	Output .obj disimpan: ..\test\cow_voxed.obj
	====================================

Output File	

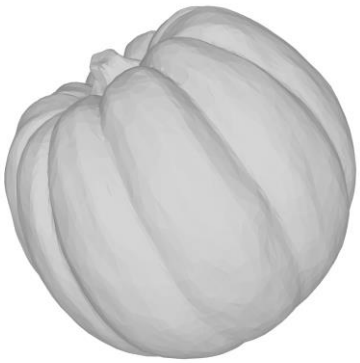
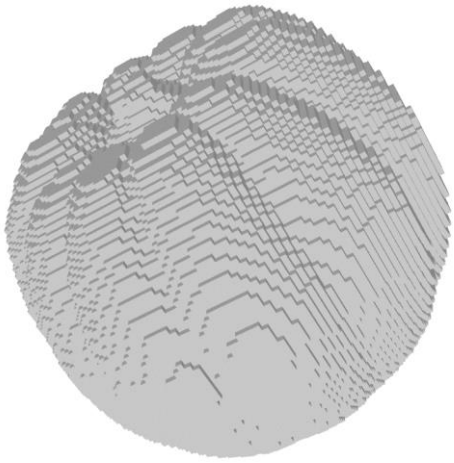
2.

Input File	

Output CLI	PS D:\stima\tucil 2\Tucil2_13524116_13524135\src> go run . cube.obj 7
	Input file : ..\test\cube.obj
	Output file: ..\test\cube_voxed.obj
	Max depth : 7
	Memproses...
	Parse selesai: 12 triangles
	Bounding box: min = (-0.000, -0.000, -0.000) size = 1.001
	Membangun octree...
	Octree selesai: 96776 voxel ditemukan
	Menulis output...
	================ Stats ================
	Jumlah voxel : 96776
	Jumlah vertices : 774208
	Jumlah faces : 1161312
	Kedalaman octree: 7
	Statistik node per depth:
	  1 : 8
	  2 : 56
	  3 : 296
	  4 : 1352
	  5 : 5768
	  6 : 23816
	  7 : 96776
	Statistik node yang dilewati per depth:
	  1 : 0
	  2 : 8
	  3 : 152
	  4 : 1016
	  5 : 5048
	  6 : 22328
	  7 : 93752
	Waktu eksekusi: 5.0712082s
	Output .obj disimpan: ..\test\cube_voxed.obj
	==
	_

Output File	

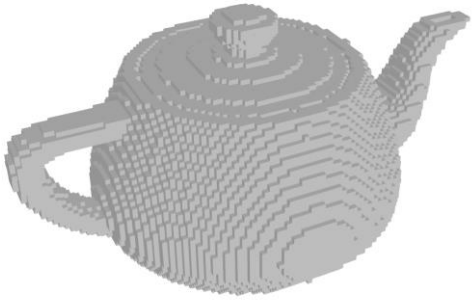
3.

Input File	
Output CLI	PS D:\stima\tucil 2\Tucil2_13524116_13524135\src> go run . pumpkin.obj 7 Input file : ..\test\pumpkin.obj Output file: ..\test\pumpkin_voxed.obj Max depth : 7 Memproses... Parse selesai: 10000 triangles Bounding box: min = (-66.688, -63.194, -174.080) size = 128.128 Membangun octree... Octree selesai: 28536 voxel ditemukan Menulis output... ================ Stats ================ Jumlah voxel : 28536 Jumlah vertices : 228288 Jumlah faces : 342432 Kedalaman octree: 7 Statistik node per depth: 1 : 8 2 : 32 3 : 122 4 : 415 5 : 1752 6 : 7089 7 : 28536 Statistik node yang dilewati per depth: 1 : 0 2 : 32 3 : 134 4 : 561 5 : 1568 6 : 6927 7 : 28176 Waktu eksekusi: 1.5117545s Output .obj disimpan: ..\test\pumpkin_voxed.obj ==========
Output File	

4.

Input File	

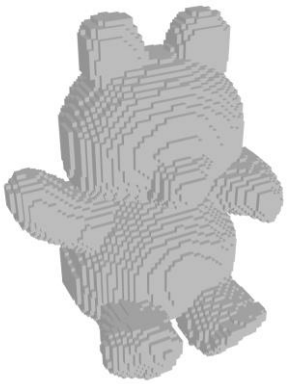
Output CLI	```
	PS D:\stima\tucil 2\Tucil2_13524116_13524135\src> go run . teapot.obj 7
	Input file : ..\test\teapot.obj
	Output file: ..\test\teapot_voxed.obj
	Max depth : 7
	Memproses...
	Parse selesai: 6320 triangles
	Bounding box: min = (-3.787, -2.429, -4.004) size = 8.008
	Membangun octree...
	Octree selesai: 19106 voxel ditemukan
	Menulis output...
	================ Stats ================
	Jumlah voxel : 19106
	Jumlah vertices : 152848
	Jumlah faces : 229272
	Kedalaman octree: 7
	Statistik node per depth:
	  1 : 8
	  2 : 16
	  3 : 64
	  4 : 292
	  5 : 1134
	  6 : 4756
	  7 : 19106
	Statistik node yang dilewati per depth:
	  1 : 0
	  2 : 48
	  3 : 64
	  4 : 220
	  5 : 1202
	  6 : 4316
	  7 : 18942
	Waktu eksekusi: 1.0234172s
	Output .obj disimpan: ..\test\teapot_voxed.obj
	==================================
	``

Output File	

5.

Input File	

Output CLI	PS D:\stima\tucil 2\Tucil2_13524116_13524135\src> go run . teddybear.obj 7
	Input file : ..\test\teddybear.obj
	Output file: ..\test\teddybear_voxed.obj
	Max depth : 7
	Memproses...
	Parse selesai: 3192 triangles
	Bounding box: min = (-32.032, -32.032, -32.032) size = 64.064
	Membangun octree...
	Octree selesai: 18564 voxel ditemukan
	Menulis output...
	================ Stats ================
	Jumlah voxel : 18564
	Jumlah vertices : 148512
	Jumlah faces : 222768
	Kedalaman octree: 7
	Statistik node per depth:
	  1 : 8
	  2 : 22
	  3 : 69
	  4 : 281
	  5 : 1146
	  6 : 4594
	  7 : 18564
	Statistik node yang dilewati per depth:
	  1 : 0
	  2 : 42
	  3 : 107
	  4 : 271
	  5 : 1102
	  6 : 4574
	  7 : 18188
	Waktu eksekusi: 967.3746ms
	Output .obj disimpan: ..\test\teddybear_voxed.obj
	==========

Output File	

4. Kesimpulan

Program yang telah dibuat berhasil melakukan proses *voxelization* pada objek 3 dimensi menggunakan struktur octree dengan baik. Algoritma yang digunakan adalah *divide and conquer*, di mana ruang tiga dimensi secara rekursif dibagi menjadi delapabn bagian (sub-ruang), lalu setiap bagian diproses secara independent hingga mencapai kedalaman tertentu.

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Varistha Devi

Eliana Natalie Widjojo

Lampiran

1. Tabel pengecekan:

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan concurrency	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

2. Pranala *repository*: https://github.com/Beeziebloop/Tucil2_13524116_13524135